

Projekt Gruppe 6 - Einführung in Software Engineering

Projekt Name: Hatch-Heroes

Projekt Inhalt (grobe Definition):

Du bekommst ein Ei, dieses musst du ausbrüten, dich um das Tier kümmern und es mit „love points“ oder einem „affection meter“ glücklich halten damit es sich schlussendlich weiterentwickelt. Wenn das Tier genug entwickelt ist, kann man gegen andere „böse“ Tiere kämpfen und Geld verdienen.

Requirements:

Players hatch a creature from an egg and care for it by feeding, washing, playing with it, and letting it sleep. Good care increases its happiness and allows it to evolve. After its **second evolution**, the creature can fight hostile creatures in turn-based battles. There are **no levels or experience points**: development depends on care and happiness.

The game uses a modern graphical interface with an animated creature, care buttons, status bars, and a separate battle screen.

Victoria Hausegger

Functional:

FR1.1

The player shall be able to name an egg and hatch it after a displayed countdown. The interface shall show the resulting creature and its current development stage.

FR1.2

The player shall be able to feed, wash, entertain, and put the creature to sleep. Each action shall improve the corresponding need—fullness, cleanliness, entertainment, or rest—displayed on a scale from 0 to 100. These values shall decrease over time while the game is running.

Non-Functional:

NFR1.1

Each of the four care actions shall be accessible within a maximum of two clicks or taps from the main pet screen.

Erol Bilyalov

Functional:

FR2.1

The system shall store the creature's name, needs, development stage, and evolution progress in a database after every completed action. These values shall be restored when the game is reopened.

FR2.2

The system shall calculate happiness from the four needs and automatically evolve the creature when the defined happiness and duration conditions are met. Evolution shall follow the sequence Egg → baby → first evolution → second evolution, without levels or experience points only according to happiness.

Non-Functional:

NFR2.1

With 10,000 stored battle-history entries, loading the current creature and its 20 most recent battles shall take no longer than two seconds on the agreed project test computer.

Jan Magbual

Functional:

FR3.1

The system shall unlock battles after the second evolution. Battles shall alternate between player and opponent turns, offer attack and defend actions, display both creatures' health, and end when either creature reaches zero health.

FR3.2

The system shall calculate damage using the chosen action, the creature's happiness at the start of the battle, and the opponent's defence. It shall select opponent actions using predefined rules and save each battle's opponent, date, and result in the database.

Non-Functional:

NFR3.1

During 100 test battles on the agreed project test computer, at least 95% of battle actions shall produce their calculated result within 200 milliseconds, excluding animation time.

Algorithm description

Happiness calculation algorithm

The game calculates the creature's happiness from its four current need values.

Let:

- **F** = Fullness, from 0 to 100
- **C** = Cleanliness, from 0 to 100
- **E** = Entertainment, from 0 to 100
- **R** = Rest, from 0 to 100

The happiness value **H** is calculated as:

$$H = \text{round}((F + C + E + R) / 4)$$

The resulting happiness value is therefore between **0 and 100**.

The calculation is repeated whenever one of the four need values changes.

Evolution algorithm

Evolution is based only on happiness and the amount of continuous time for which the required happiness threshold is maintained.

Egg → Baby

1. The player assigns a valid name to the egg.
2. A 60-second countdown begins.
3. When the countdown reaches zero, the development stage changes to Baby.

Baby → First Evolution

1. The system continuously evaluates happiness.
2. If happiness is **70 or higher**, the evolution timer progresses.
3. If happiness falls below **70**, the timer is reset to zero.
4. If happiness remains at least 70 for **5 continuous minutes**, the creature evolves to First Evolution.

First Evolution → Second Evolution

1. The system continuously evaluates happiness.
2. If happiness is **80 or higher**, the evolution timer progresses.
3. If happiness falls below **80**, the timer is reset to zero.
4. If happiness remains at least 80 for **10 continuous minutes**, the creature evolves to Second Evolution.

No character level or experience-point calculation is involved in this algorithm.

Battle damage algorithm

At the beginning of a battle, the player's current happiness value is stored as the battle happiness value **H**.

For an **Attack** action, damage is determined using:

- a base damage value of **20**;
- the player's happiness at the beginning of the battle;
- the opponent's defence value.

Each hostile creature has a predefined defence value **D** in the range **0 to 100**.

Damage is calculated as:

$$\text{Damage} = \text{round}(20 \times (0.5 + H / 200) \times (1 - D / 200))$$

where:

- **H** is the player's battle happiness from 0 to 100;
- **D** is the defending creature's defence value from 0 to 100.

The calculated damage is subtracted from the target's health.

A creature's health cannot fall below zero.

Defend action

When a creature selects **Defend**, the damage received from the next successful Attack action against that creature is reduced by **50%**, rounded to the nearest whole number.

The defence effect expires immediately after reducing one incoming Attack action.

Opponent-action algorithm

The opponent uses predefined deterministic rules rather than player input.

1. If the opponent's current health is greater than **30**, the opponent selects **Attack**.
2. If the opponent's current health is **30 or lower**, the opponent selects **Defend**.
3. After the opponent action has been processed, control returns to the player unless one creature has reached zero health.

This algorithm ensures that opponent behaviour is reproducible and can be verified in automated tests.

Battle termination algorithm

After every action that can cause damage, the system evaluates both creatures' health.

- If the opponent reaches **0 health first**, the battle ends with the result **Win**.
- If the player's creature reaches **0 health first**, the battle ends with the result **Loss**.

After the battle ends, the system stores the opponent, completion date and time, and battle result in the database.

A victory increases the player's currency balance by **100 coins**.

The completed battle is then added to the battle history.

Requirements of data handling

DH1 – Creature identification

The system shall store exactly one player-defined name for the currently active creature.

Verification: Create a creature with a defined name, close and reopen the application, and verify that the same name is displayed.

DH2 – Creature needs

The system shall store the current values for **fullness, cleanliness, entertainment, and rest** as integer values in the range **0 to 100 inclusive**.

Verification: Inspect the persisted creature record and confirm that all four values are present and within the permitted range.

DH3 – Development stage

The system shall store the creature's current development stage as one of the following four values:

Egg, Baby, First Evolution, Second Evolution.

Verification: Inspect the persisted value after each evolution.

DH4 – Evolution progress

The system shall store the continuous duration for which the current evolution happiness condition has been satisfied.

Verification: Maintain the required happiness value, restart the application, and verify that the previously accumulated evolution duration is restored.

DH5 – Player currency

The system shall store the player's current in-game currency balance as a non-negative integer.

Verification: Win a battle, restart the application, and compare the restored currency balance with the balance before closing the application.

DH6 – Battle history opponent

For every completed battle, the system shall store the identifier or name of the opponent.

Verification: Complete a battle and inspect the corresponding battle-history record.

DH7 – Battle history timestamp

For every completed battle, the system shall store the date and time at which the battle ended.

Verification: Complete a battle and compare the stored timestamp with the actual completion time.

DH8 – Battle history result

For every completed battle, the system shall store exactly one result with the value **Win** or **Loss**.

Verification: Complete one winning and one losing battle and verify the persisted values.

DH9 – Persistence after care actions

The system shall persist all creature-state values affected by a completed care action no later than **1 second after the action has completed**.

Verification: Perform each care action individually and inspect the database within one second after completion.

DH10 – Persistence after evolution

The system shall persist the new development stage no later than **1 second after an evolution has occurred**.

Verification: Trigger an evolution and inspect the persisted development stage.

DH11 – Restoration after application restart

When the game is reopened, the system shall restore the most recently persisted creature name, need values, development stage, evolution progress, and player currency before the main pet screen becomes interactive.

Verification: Modify the stored state, close the application, reopen it, and compare all restored values with the most recently persisted values.

DH12 – Battle-history capacity

The database shall support at least **10,000 completed battle-history records** without deleting existing records automatically.

Verification: Populate the database with 10,000 battle records and verify that all records remain accessible.